

Article

Machine Learning Approaches for Early Student Performance Prediction in Programming Education

Seifeddine Bouallegue ^{1,*} , Aymen Omri ¹  and Salem Al-Naemi ² 

¹ Department of Software Systems, University of Doha for Science and Technology, Doha 24449, Qatar; aymen.omri@udst.edu.qa

² College of Computing and IT, University of Doha for Science and Technology, Doha 24449, Qatar; salem.alnaemi@udst.edu.qa

* Correspondence: seifeddine.bouallegue@udst.edu.qa

Abstract

Intelligent recommender systems are essential for identifying at-risk students and personalizing learning through tailored resources. Accurate prediction of student performance enables these systems to deliver timely interventions and data-driven support. This paper presents the application of machine learning models to predict final exam grades in a university-level programming course, leveraging multi-modal student data to improve prediction accuracy. In particular, a recent raw dataset of students enrolled in a programming course across 36 class sections from the Fall 2024 and Winter 2025 terms was initially processed. The data was collected up to one month before the final exam. From this data, a comprehensive set of features was engineered, including the student's background, assessment grades and completion times, digital learning interactions, and engagement metrics. Building on this feature set, six machine learning prediction models were initially developed using data from the Fall 2024 term. Both training and testing were conducted on this dataset using cross-validation combined with hyperparameter tuning. The XGBoost model demonstrated strong performance, achieving an accuracy exceeding 91%. To assess the generalizability of the considered models, all models were retrained on the complete Fall 2024 dataset. They were then evaluated on an independent dataset from Winter 2025, with XGBoost achieving the highest accuracy, exceeding 84%. Feature importance analysis has revealed that the midterm grade and the average completion duration of lab assessments are the most influential predictors. This data-driven approach empowers instructors to proactively identify and support at-risk students, enabling adaptive learning environments that deliver personalized learning and timely interventions.

Keywords: early prediction; machine learning; personalized learning; programming education; student performance



Academic Editors: Chien-Hung Lai
and Gholamreza Anbarjafari

Received: 2 December 2025

Revised: 31 December 2025

Accepted: 5 January 2026

Published: 8 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Artificial Intelligence (AI) and Machine Learning (ML) have rapidly expanded across multiple fields [1–5]. They have demonstrated strong capabilities in addressing complex prediction and decision-making problems. Recent research efforts highlight their impact on cybersecurity and malware detection [1,2], higher education and digital learning environments [3], university performance forecasting [6], political voting prediction [7], and financial market analysis [8].

These advancements not only illustrate the maturity and versatility of modern AI techniques but also create a strong foundation for leveraging similar approaches within educational settings. Building on this broader progress, the growing integration of technology and AI tools into education has given rise to smart educational systems designed to enhance and personalize student learning [9,10]. Among these, educational recommender systems have demonstrated strong potential in guiding learners toward resources and activities tailored to their individual needs [11,12]. This personalization is particularly crucial in programming courses, where many students encounter difficulties with abstract reasoning and problem-solving [13]. Such challenges often hinder early progress, underscoring the importance of promptly identifying at-risk students [14]. In these contexts, timely and personalized interventions can play a pivotal role in improving learning outcomes and reducing dropout rates—especially in large-scale or online learning environments where individualized support is inherently limited [10,15].

Despite the abundance of student interaction data available through learning management systems and integrated development environments, many educational platforms still lack effective mechanisms for early performance prediction [9,11,16]. In the absence of predictive insights, recommender systems struggle to adapt dynamically to individual learning trajectories, thereby limiting their capacity to provide timely support for at-risk students. Recent research has emphasized the value of combining artificial intelligence with gamification to enhance engagement and personalization in education. Systems like KINAITICS [17] have shown how intelligent, gamified interventions can significantly improve motivation and learning outcomes.

Therefore, there is a pressing need to develop predictive models capable of forecasting students' final performance at an early stage of the course, enabling timely identification of at-risk students and the implementation of appropriate pedagogical interventions.

Previous research has explored various approaches to predicting student performance, drawing on academic records, behavioral patterns, and interaction data from Learning Management Systems (LMS). However, most of these studies fall short of leveraging multi-modal feature sets in a comprehensive manner for early performance prediction. A multi-modal approach would enable the development of more effective personalized recommender systems, capable of delivering timely and targeted interventions—an aspect that is particularly crucial in programming education, where early difficulties can significantly hinder learning progress.

1.1. Paper Contributions

In this work, we propose a machine learning framework for early prediction of student performance in a university-level programming course. The framework leverages multi-modal student data to support intelligent educational recommender systems. In particular, the key contributions of this work are as follows:

- The design of a comprehensive feature engineering pipeline that integrates diverse data sources, including students' background information, assessment grades and completion times, digital learning platform interactions, and engagement metrics.
- A comparative evaluation of six machine learning models trained on Fall 2024 data, with extensive use of cross-validation and hyperparameter tuning. The XGBoost model achieved the highest predictive accuracy, exceeding 91%.
- A generalization study across academic terms, where models trained on Fall 2024 were tested on an independent Winter 2025 dataset. XGBoost consistently outperformed other models, reaching an accuracy above 84%.

- A feature importance analysis highlighting midterm exam grades and average completion duration of lab assessments as the most influential predictors of final exam performance.
- An exploration of how accurate early-grade prediction can enhance intelligent educational recommender systems, emphasizing their potential to enable timely and personalized interventions for at-risk students.

Our experimental findings, derived from a dataset comprising 36 class sections across two academic terms, demonstrate that the proposed predictive approach reliably forecasts final exam performance using data available one month prior to the final exam. These results underscore the potential of integrating predictive models into educational recommender systems to empower instructors with data-driven insights, foster student engagement, and improve academic outcomes in computing disciplines.

1.2. Paper Organization

The remainder of this paper is organized as follows. Section 2 reviews prior research on early performance prediction in programming courses using ML techniques. Section 3 presents the main contribution of this work, outlining the key steps of the proposed ML framework for predicting final exam grades. Section 4 reports and analyzes the experimental results. Finally, Section 5 concludes the paper and outlines directions for future research.

2. Related Works

The field of educational data mining has seen a surge in research focused on predicting student performance to facilitate early interventions and personalized learning [18,19]. Particularly in programming education, where students often face significant challenges, early prediction models are crucial for developing effective recommender systems and support mechanisms [20].

In this context, and as summarized in Table 1, although a substantial body of research has applied ML techniques to student performance prediction, only a limited number of studies have specifically focused on programming education courses using multimodal LMS data and cross-term temporal validation.

Indeed, in [21], Nabil et al. examined the prediction of student performance in data structures courses. To achieve this, they applied Deep Neural Networks (DNN), Random Forests (RF), and Support Vector Classifiers (SVC). The models relied on historical grades and records of failed courses. As a result, the study contributed to the early detection of at-risk students. However, its scope remained limited due to the absence of multi-modal LMS data.

In the context of programming courses, Alnoman et al. [22] applied Support Vector Machines (SVM), Bagged Trees, and K-Nearest Neighbors (KNN) to classify students in the UAE according to whether they achieved a grade of 90% or higher. Bagged Trees and KNN showed strong performance. However, the study was restricted to binary classification and did not address the early identification of at-risk students.

In [23], Alhazmi et al. focused on early detection by leveraging institutional academic records with RF, XGBoost, and SVM models. Their work examined performance trends but did not incorporate multi-modal LMS data. Similarly, Kinjal et al. [24] applied a hybrid regression model across 56 programs in Saudi Arabia, using GPA history and course completion ratios from both university databases and the Open University Learning Analytics Dataset (OULAD). Although the model achieved strong performance, it excluded multi-modal LMS data and was not deployed in real classroom settings.

Table 1. Related works to the topic of early prediction of students' final grades in a programming course.

Refs.	Course	Country	ML Model	Features	Evaluation Metric	Data Source	Advantages and Limitations
[21]	Data Structures	Egypt	DNN, RF, and SVC	Grades from previous courses, and number of failed courses	Accuracy, and Classification error	University Database	+ Early identification of students at risk. + Used cross validation and statistical tests to support the results – The paper did not use multi-model LMS data.
[22]	Program- ming	UAE	SVM, Bagged Trees, and KNN	High School GPA, Midterm Grade, and Absence Percentage.	Accuracy, Precision, Recall, and F1-Score.	University Database.	+ High accuracy across the models used, especially for bagged trees and KNN. – The paper was only focused on the student getting an A or not. – The paper did not consider early identification of students at risk.
[23]	First-year courses	Saudi Arabia	RF, XGBoost and SVM	Admission Scores, Gender, and First Year Course Grades	Accuracy, Precision, Recall, F1-Score, and Confusion Matrix	University Database	+ The paper predicts the final performance early. – No LMS data was used in this work.
[24]	multi- program: 56 courses across different programs	Saudi Arabia	Hybrid Regression Model	Student GPA History, and Course Completion Ratio.	Root Mean Squared Error.	University Database, and OULAD.	+ Combination of multiple models to predict the grades and their causation. – Multi-modal LMS data not considered. – The model was not used in real classrooms.
[14]	Programming and Computational Thinking	Spain and Colombia	SVM, and GB	Results of Motivated Strategies for Learning Questionnaire	Precision, Recall, and F1-Score	University database and survey data from 11 participating students	+ This paper uses weekly predictions for student feedback, and explores which features are the most contributing through feature importance. – A very small sample size with only 11 students.
[25]	Introduction to Programming, Intermediate Programming, Data Structures, and Cybersecurity	USA, Canada, and Germany	LR, and GB	Behavioral data from IDE plugin: Time Spent on Exercises, Number of Errors, and Time between Sessions)	Accuracy, Precision, Recall, and F1-Score.	IDE Data	+ The paper was able to early predict the performance of the students – The paper did not consider multi-modal data, e.g., previous assessment grades and duration.
[26]	Introductory Programming	USA	KNN, and XGBoost	Behavioral data from CodeWorkOut platform: Time Spent, Correct Submissions, Incorrect Submissions, Compile Errors	Root Mean Squared Error, and R-squared	Dataset from CodeWorkOut	+ The paper was primary focused on the coding behavior of the students – The work did not filter out the submission which could lead to plagiarism – The work did not consider early prediction of student at risk.

KNN: K-Nearest Neighbors, SVM: Support Vector Machine, , LR: Linear Regression, OULAD: Open University Learning Analytics Dataset, DNN: Deep Neural Network, RF: Random Forest, GB: Gradient Boosting, SVC: Support Vector Classifier, IDE: Integrated Development Environment.

Another study, presented in [14], involved Motivated Strategies for Learning Questionnaire (MSLQ) data from students in Spain and Colombia enrolled in programming and computational thinking courses. The authors used SVM and Gradient Boosting to generate weekly predictions and performed feature importance analysis. However, the study was severely limited by a very small sample of only 11 students.

In [25], behavioral features from programming environments were investigated. The study relied on Integrated Development Environment (IDE) plugin logs—such as coding time spent coding and error patterns—to predict student performance across institutions in the USA, Canada, and Germany. Logistic Regression (LR) and Gradient Boosting (GB) models were applied, yielding good predictive results. However, the work did not include multi-modal data, such as prior assessment grades or task durations.

A different work by Hoq et al. [26] focused on coding behavior captured through the CodeWorkOut platform in the USA. The authors employed KNN and XGBoost to predict outcomes based on submission patterns and compile errors. However, this work did not filter out potentially plagiarized submissions and did not address the early prediction of at-risk students.

While previous studies and research works have investigated and introduced different approaches for predicting student performance, the integration of multi-modal features for early prediction remains underutilized. This limitation reduces the ability of personalized recommender systems to deliver timely support, particularly in programming education.

Recently, we attempted to address this gap in [27] by integrating academic assessments, engagement metrics, and behavioral data to build an early prediction framework. While this initial work demonstrated the feasibility of predicting student performance with high accuracy, it was limited to a single term and did not fully evaluate model generalizability across cohorts. In this work, we significantly extend that study by leveraging a larger multi-term dataset and applying more rigorous training and validation procedures, including cross-validation with hyperparameter tuning and evaluation on an independent dataset from a different term. In contrast to many existing models, we emphasize early prediction using data available one month before the final exam to provide actionable insights for instructors. Furthermore, by combining robust model evaluation with explainability through feature importance analysis, our approach advances the link between predictive analytics and practical deployment in educational support systems.

3. Machine Learning-Based Early Performance Prediction

This section details the ML process used to predict final exam grades in a university-level programming course. As illustrated in Figure 1, key steps of the ML process used in this work are illustrated, including the following:

3.1. Problem Definition

The problem addressed in this work is the early prediction of students' final exam (FE) grades in a university-level programming course. Given a set of multi-modal features—such as academic records, assessment outcomes, learning behavior, and platform interaction logs—the goal is to accurately estimate each student's final performance. The main challenge lies in leveraging heterogeneous and incomplete data to build a predictive model that generalizes well across diverse student groups and academic terms. In this context, accurate early predictions empower instructors to identify at-risk students in advance. This, in turn, enables timely and personalized interventions that can enhance learning outcomes and improve student retention.

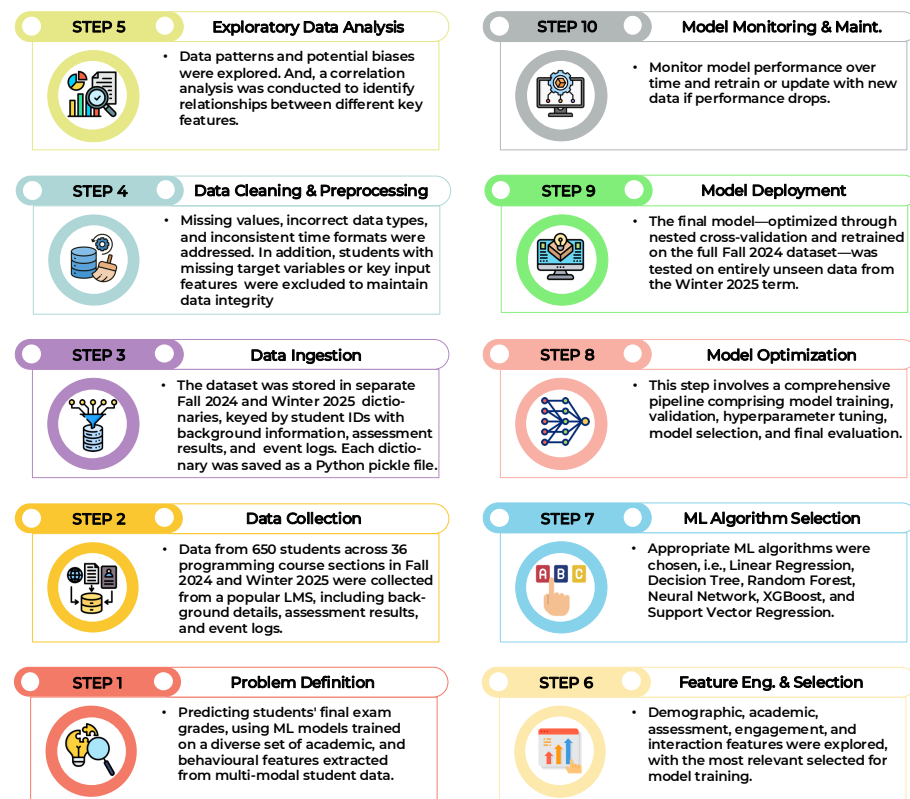


Figure 1. Key steps of ML process used to predict final exam grades in a university-level programming course.

3.2. Data Collection

Data collection was conducted during the Fall 2024 and Winter 2025 terms, involving 650 university students enrolled in a programming course across 36 class sections. The dataset was extracted from a widely used Learning Management System (LMS) and includes various types of information, such as student background details, assessment results, and detailed event logs. Initially, the data were stored in separate CSV files before being processed.

3.3. Data Ingestion

The dataset was loaded into dictionary objects keyed by student IDs, with each entry containing three main components: background information, assessment results, and event logs. Separate dictionaries were maintained for the Fall 2024 and Winter 2025 terms. This structure allowed for efficient access and organization of individual student data. Each dictionary was initially saved as a serialized Python pickle file to preserve its structure and enable quick loading for subsequent processing and analysis.

3.4. Data Cleaning and Preprocessing

During this preprocessing step, missing values were addressed, incorrect data types were corrected, and inconsistent time formats were standardized. Additionally, students lacking the considered variables—such as final exam grades or key input features—were excluded to preserve data integrity and ensure the reliability of subsequent analysis and modeling.

3.5. Exploratory Data Analysis

Data patterns and relationships were explored, and trends were visualized to detect potential biases. To support this analysis, descriptive statistics were computed to reveal

grade distributions and interaction patterns. Furthermore, a correlation matrix was used to identify relationships between different features.

Patterns and relationships within the data were examined, while visualizations were employed to highlight trends and uncover potential biases. To complement this analysis, descriptive statistics were calculated to present grade distributions and interaction patterns. In addition, a correlation matrix was utilized to identify relationships among the different features.

In particular, we present in Figure 2a,b the correlation matrices between key features extracted from the cleaned datasets for the Fall 2024 and Winter 2025 terms. These matrices offer a comprehensive view of the linear relationships between background variables, assessment results, engagement behaviors, and final exam performance.

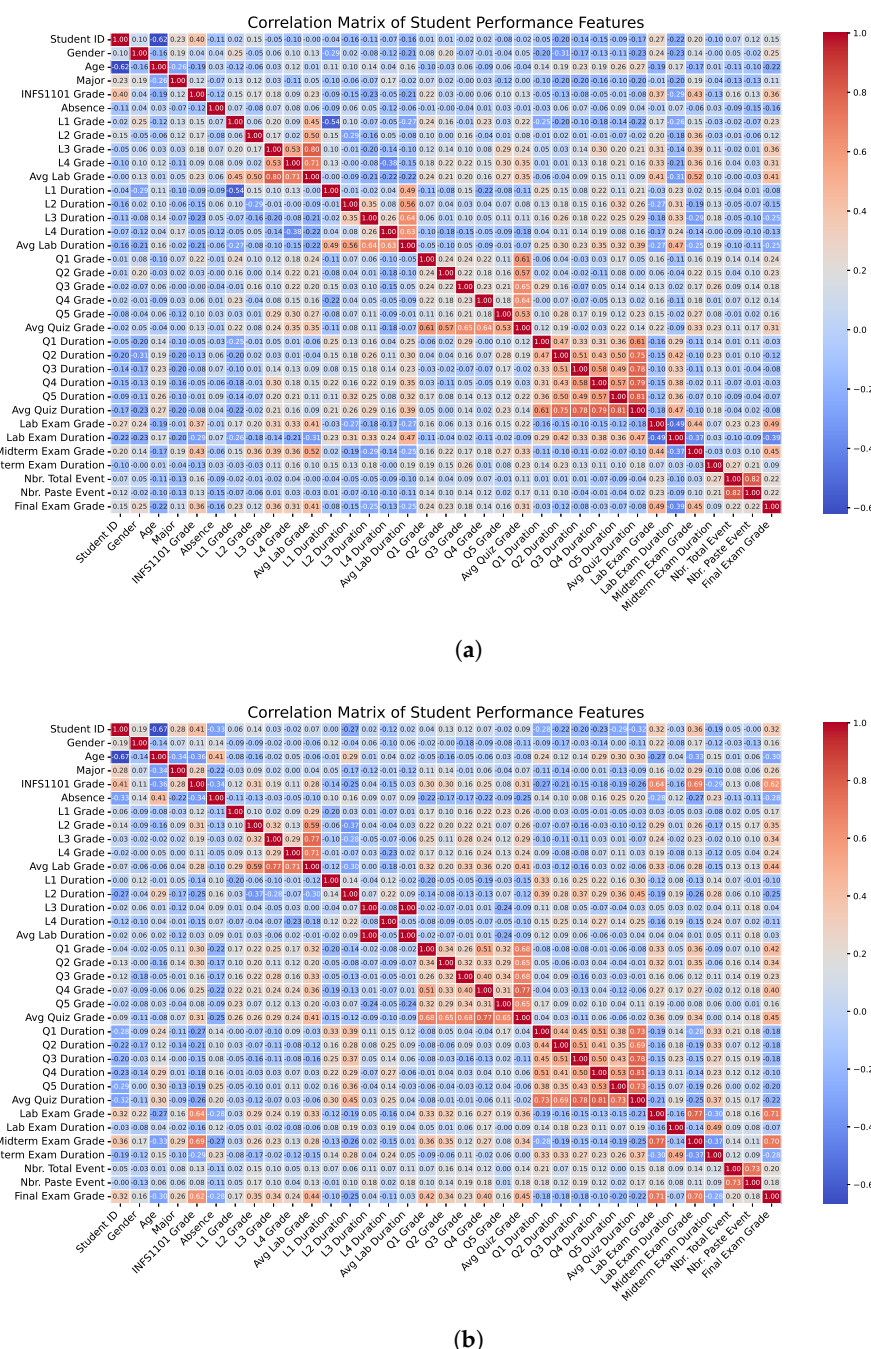


Figure 2. Correlation matrices for (a) the Fall 2024 term, and (b) the Winter 2025 term.

As illustrated in both matrices, academic performance features—such as Lab Exam Grade, Midterm Exam Grade, and selected Quiz Grades (particularly Q3, Q4, and Q5)—demonstrate the strongest positive correlations with Final Exam Grade. This consistent pattern across terms reinforces the role of prior academic success as a reliable predictor of final outcomes. Grades from earlier assessments, such as L1–L4 and Q1–Q2, also exhibit moderate positive correlations, indicating that academic progression over the term contributes cumulatively to final performance.

In contrast, duration-based features—such as time spent on labs, quizzes, and exams—show generally weaker and occasionally negative correlations with Final Exam Grade. These findings suggest that longer task durations may not directly reflect stronger engagement or understanding; in some cases, they may even indicate struggle or inefficiency. Similarly, behavioral indicators derived from platform interaction logs (e.g., number of paste events and total events) present weak or slightly negative correlations, implying that raw activity metrics alone offer limited explanatory power regarding academic achievement.

Taken together, these results highlight the importance of performance-based features in predictive modeling, while suggesting that behavioral and engagement data, although less predictive on their own, may still contribute value when integrated with other features. Such insights inform the design of hybrid models that combine cognitive and behavioral dimensions to more accurately identify at-risk students and support early intervention strategies.

3.6. Feature Engineering and Selection

To build effective predictive models, a comprehensive set of features was engineered from the raw dataset. These features include student demographics, historical academic performance, detailed assessment outcomes, engagement metrics, and interaction durations. In particular, the following key features were selected for further analysis:

- **Student ID:** A unique identifier for each student. Although not used directly in modeling, it enables student-level data tracking. Note that lower ID values often correspond to older student cohorts.
- **Gender, Age, Major:** Background attributes that provide demographic context and can influence learning patterns.
- **INFS1101 Grade:** The student's grade in a prerequisite introductory course, used as a prior academic performance indicator.
- **Lab 1–Lab 4 Grades, Avg Lab Grade:** Grades obtained in four lab assessments, with the average summarizing overall lab performance. These assessments were conducted during the Fall 2024 term within one month of the final exam.
- **Lab 1–Lab 4 Durations, Avg Lab Duration:** Time spent completing each lab, along with the average, used to capture behavioral engagement during assessments.
- **Quiz 1–Quiz 5 Grades, Avg Quiz Grade:** Scores in five in-term quizzes, and their average. As with labs, only quizzes conducted in the final month before the exam were considered.
- **Quiz 1–Quiz 5 Durations, Avg Quiz Duration:** Duration metrics reflecting time spent on each quiz and the average, which can indicate level of effort or engagement.
- **Lab Exam Grade, Lab Exam Duration:** Performance and completion time for the lab exam, held prior to the final exam.
- **Midterm Exam Grade, Midterm Exam Duration:** Grade and time taken for the midterm exam, an important milestone assessment one month prior to the final exam.
- **Nbr. Total Event:** Total number of interactions (e.g., running code, viewing materials, submitting work) recorded in the LMS within the last month of the term.

- Nbr. Paste Event: Subset of events representing paste actions, often interpreted as potential indicators of academic behavior (e.g., reuse or copying of code).
- Final Exam Grade: The target variable to be predicted, representing the student's final performance in the course.

These features collectively capture a blend of historical knowledge, in-course progression, behavioral signals, and assessment outcomes to enable robust modeling of final exam performance.

3.7. ML Algorithm Selection

This step involves identifying and selecting the most suitable ML algorithms (model types) for the defined prediction problem. The choice is guided by the nature of the task (e.g., regression or classification), the structure and dimensionality of the data, and prior knowledge about model interpretability and performance. Candidate algorithms are then subjected to further training, validation, and tuning.

In this work, multiple regression models were considered to assess predictive performance. These models were chosen based on their robustness and ability to handle complex relationships in the considered data. In the following more details about these model are presented:

- Linear Regression: A simple linear model with scaled input features using MinMaxScaler, which assumes a linear relationship between the predictors and the final exam grade. The relevant coefficients indicate feature importance based on absolute values.
- Decision Tree Regressor: A non-linear model that splits the data based on feature thresholds to predict final grades without requiring input scaling. The tree structure allows capturing complex decision boundaries.
- Random Forest Regressor: An ensemble of decision trees trained on different data subsets to improve robustness and accuracy. It aggregates the predictions of all trees and provides feature importance scores based on their contribution across trees.
- XGBoost Regressor: A gradient boosting model that builds decision trees sequentially, with each tree aimed at correcting the errors of its predecessors. It is known for its efficiency and predictive accuracy, using a specified number of trees and learning rate.
- Neural Network (Keras Sequential Model): A feedforward neural network composed of an input layer, a hidden layer with a given activation function, and a single output node. The model is trained using a standard optimizer and employs early stopping to prevent overfitting.
- Support Vector Regression (SVR): A kernel-based model trained on scaled data to capture non-linear relationships. It fits a function within a specified margin of tolerance to predict continuous outcomes, making it suitable for regression tasks with complex patterns.

3.8. Model Optimization

The model optimization stage involves a comprehensive pipeline comprising model training, validation, hyperparameter tuning, model selection, and final evaluation. To ensure robust performance and minimize overfitting, a nested k -fold cross-validation framework was adopted. In this setup, the inner loop is dedicated to validation and hyperparameter tuning, while the outer loop is used for unbiased performance evaluation on unseen data. This structured approach allows for the selection of the best-performing model configuration while ensuring reliable estimates of generalization performance.

3.8.1. Phase 1: Model Optimization via Nested Cross-Validation

In the first phase, model optimization was performed using data exclusively from the Fall 2024 term. A nested k -fold cross-validation setup was employed to systematically evaluate and tune multiple candidate models. Specifically, a 5-fold outer loop was used for model evaluation, while within each outer training split, a 3-fold inner cross-validation was used to perform hyperparameter tuning via randomized search. For each candidate model, a set of hyperparameter combinations was sampled across 100 iterations.

Feature scaling was applied where required using *StandardScaler*, fitted within each training fold to prevent data leakage. Models such as Support Vector Regression, Neural Network, and Linear Regression were trained on scaled features. For each outer fold, the best-performing model obtained from the inner-loop validation was evaluated on the corresponding outer test fold. Accuracy is reported as the primary evaluation metric to ensure clarity and interpretability for an educational research audience. Additional metrics, including MSE and R^2 , are reserved for future extended work, where a more comprehensive multi-metric evaluation will be conducted.

The detailed optimized hyperparameter configurations for all considered ML models are provided in Appendix A.

3.8.2. Phase 2: Model Retraining on Full Training Data

In this phase, the best model configuration obtained from the nested cross-validation process was retrained using the complete Fall 2024 dataset. This final model, incorporating the optimal hyperparameters selected during tuning, was then evaluated on unseen data to assess its generalization performance. Specifically, the model was tested on an independent dataset from the Winter 2025 term, which was not used in either training or hyperparameter tuning. This cross-term evaluation strategy provides a strong and realistic assessment of robustness, helping to mitigate overfitting risks and demonstrating the model's ability to generalize beyond the original training cohort.

3.9. Model Deployment

The final model—optimized through nested cross-validation and retrained on the full Fall 2024 dataset—was tested on entirely unseen data from the Winter 2025 term. This final testing phase reflects a realistic deployment scenario, in which a model trained on historical data is applied to predict outcomes for a new student cohort.

Although the model was not integrated into a live production system, this evaluation serves as an effective proxy for deployment, providing insights into its generalization performance under temporal shifts. The trained models are fully encapsulated and ready for integration into an academic performance monitoring system. Outputs such as predicted final exam scores and ranked feature importances can assist instructors in identifying at-risk students and designing timely, targeted interventions.

3.10. Model Monitoring and Maintenance

Monitoring strategies involve periodic re-evaluation of model performance using fresh student data and retraining as needed to account for curriculum changes or evolving student behavior. Feature drift and prediction accuracy can be tracked using error metrics and updated feature importance plots.

This step is part of our future work and outlines a prospective direction for this research work.

4. Results and Discussion

Figure 3 presents a comparative analysis of the estimation accuracy of six machine learning models in predicting students' final exam grades, using data from the Fall 2024 term. The actual grades are represented by a dashed blue line with circular markers, while the estimated grades for each model are overlaid in red. The evaluated models include XGBoost, Linear Regression, Random Forest, Support Vector Regression (SVR), Decision Tree, and a Neural Network. Among these, the XGBoost model achieved the highest accuracy (91.24%), which outperforms comparable approaches reported in the literature, including [14,25], closely followed by Linear Regression (90.09%), Random Forest (89.63%), and SVR (89.61%). In contrast, the Neural Network and Decision Tree models yielded slightly lower accuracies at 82.31% and 88.81%, respectively. These results indicate that ensemble-based methods (XGBoost and Random Forest) and regularized linear approaches (Linear Regression) provide more reliable grade estimates than standalone models, such as Decision Trees and shallow neural networks.

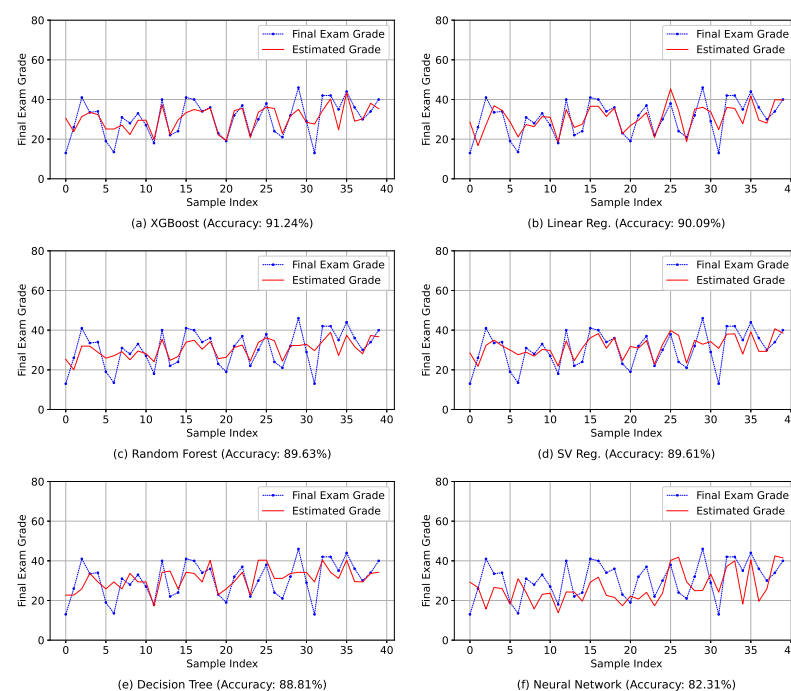


Figure 3. Prediction Accuracies of ML Models on Fall 2024 Term Data.

To assess generalization capability, the final models trained on the full Fall 2024 dataset were evaluated on the temporally distinct Winter 2025 cohort. As shown in Figure 4, this phase simulates a realistic deployment scenario in which predictions are made for future students using previously collected data. Among the six models, the Random Forest and XGBoost regressors maintained the highest accuracies at 84.28% and 83.77%, respectively. These were followed by Decision Tree (82.31%) and Support Vector Regression (78.99%), while Linear Regression (75.71%) and particularly the Neural Network (48.99%) showed substantial declines in performance.

Compared to their training-phase performance, most models exhibited moderate decreases in accuracy, underscoring the challenge of temporal generalization in educational data. The sharp drop in neural network accuracy suggests possible overfitting or poor adaptability to changes across student cohorts. On the other hand, ensemble methods such as Random Forest and XGBoost consistently demonstrated robustness, with only marginal reductions in predictive accuracy.

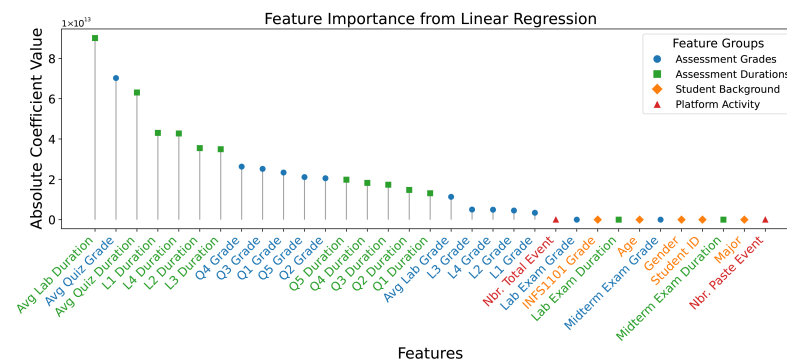


Figure 6. Feature Importance Derived from Linear Regression.

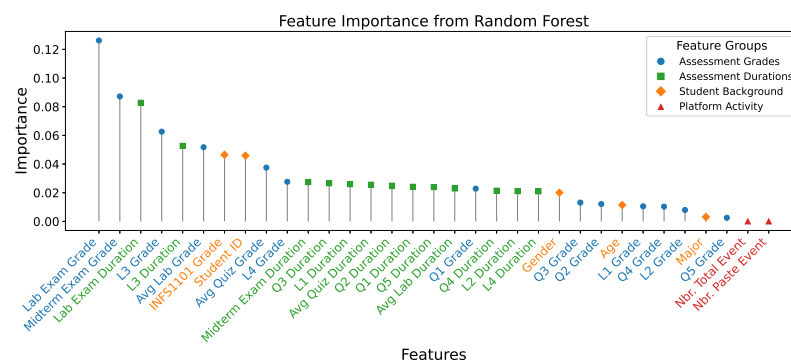


Figure 7. Feature Importance Derived from Random Forest.

In Figure 7, the Random Forest model again places Lab Exam Grade, Midterm Exam Grade, and Avg Lab Grade at the top of the importance hierarchy. This further confirms that tree-based ensemble models prioritize assessment outcomes over behavioral or demographic variables. While durations such as Lab Exam Duration and L3 Duration retain moderate influence, platform activity features exhibit very low importance.

Collectively, these results consistently demonstrate that assessment grades and durations are the dominant predictors of final performance, while features related to student demographics and behavioral engagement offer limited value.

These findings affirm the reliability of assessment-based features across cohorts, and suggest that tree-based ensemble models are better suited for deployment in real-world academic performance monitoring systems, especially when aiming for model longevity and generalization across terms.

While the results demonstrate the effectiveness and robustness of the proposed ML approach, several aspects offer opportunities for further extension and validation. The study was conducted within a single university and focused on a specific programming course, providing a controlled and realistic educational setting; future work will aim to validate the approach across additional institutions, courses, and instructional contexts. Although cross-term evaluation using independent Fall 2024 and Winter 2025 datasets supports the generalizability of the models, extending the analysis to longer academic periods would enable deeper longitudinal assessment. In addition, the current multi-modal feature set primarily captures structured academic records and learning management system interactions; future research will explore the integration of complementary data sources, such as fine-grained programming behaviors or code quality indicators, to further enrich the predictive framework. Finally, predictions were based on data collected up to one month prior to the final exam, enabling timely interventions, while future studies will investigate earlier-stage and fully real-time prediction scenarios to enhance proactive student support.

5. Conclusions

In this paper, we investigated the effectiveness of machine learning techniques in the early prediction of student performance, with the goal of supporting intelligent recommender systems in programming education. Six different ML models were evaluated using multi-modal data collected before final exams, encompassing assessment results, behavioral activity, and student background features. When evaluated on Fall 2024 data, ensemble methods—particularly XGBoost and Random Forest—demonstrated the highest predictive accuracy, exceeding 90% in estimating final exam grades. To assess model generalizability, the best-performing models were further tested on a temporally distinct cohort from Winter 2025. While all models experienced some decline in performance, Random Forest and XGBoost remained the most robust, achieving accuracies of 84.28% and 83.77%, respectively. In contrast, simpler models like Linear Regression and more sensitive architectures like Neural Networks showed substantial performance degradation, highlighting the importance of model stability in real-world deployment.

Feature importance analysis across both phases consistently identified Lab Exam Grade, Midterm Exam Grade, and Average Lab Grade as the strongest predictors of final performance. Durations of assessment completion also contributed moderately, while behavioral metrics such as event counts and paste actions had minimal predictive value. These findings provide actionable insights for educators to identify at-risk students early and deliver timely, personalized interventions.

As future work, the models can be expanded to larger and more diverse student populations across different programming courses and institutions. Further directions include deploying these models in real-time educational platforms, integrating dynamic behavioral analytics, and employing explainable AI techniques to enhance transparency and trust. Such developments will play a crucial role in advancing personalized learning strategies, ultimately improving student outcomes and retention in computer science education.

Author Contributions: Conceptualization, S.B. and A.O.; methodology, S.B. and A.O.; software, S.B.; validation, S.B., A.O. and S.A.-N.; formal analysis, S.B., A.O. and S.A.-N.; investigation, A.O. and S.A.-N.; resources, S.B.; data curation, S.B.; writing—original draft preparation, S.B.; writing—review and editing, A.O.; visualization, S.B. and A.O.; supervision, S.A.-N.; project administration, S.B. and S.A.-N.; funding acquisition, S.B. and S.A.-N. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Qatar Research Development and Innovation Council, grant number ARG01-0516-230182 and The APC was funded by Qatar Research Development and Innovation Council.

Institutional Review Board Statement: The study was conducted in accordance with the Declaration of Helsinki, and approved by the Institutional Review Board of Vice-Chair, PHCC Institutional Review Board (protocol code BUHOOTH-D-23-00091 and date of approval 8 September 2025).

Informed Consent Statement: Informed consent was obtained from all subjects involved in the study.

Data Availability Statement: The data is owned by the University of Doha for Science and Technology, and no permission has been granted to publish or distribute it.

Acknowledgments: Research reported in this publication was supported by the Qatar Research Development and Innovation Council [ARG01-0516-230182].

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Optimized Hyperparameter Configurations for the Considered ML Models

Table A1. Linear Regression Model.

Parameter	Value
copy_X	True
fit_intercept	True
n_jobs	None
positive	False

Table A2. SV Regression Model.

Parameter	Value
C	0.1
cache_size	200
coef0	0.0
degree	3
epsilon	1.0
gamma	auto
kernel	linear
max_iter	−1
shrinking	True
tol	0.001
verbose	False

Table A3. Neural Network Model.

Parameter	Value
activation	relu
alpha	0.001
batch_size	auto
beta_1	0.9
beta_2	0.999
early_stopping	True
epsilon	1×10^{-8}
hidden_layer_sizes	[128, 128]
learning_rate	constant
learning_rate_init	0.001
max_fun	15,000
max_iter	5000
momentum	0.9
n_iter_no_change	10
nesterovs_momentum	True
power_t	0.5
random_state	42
shuffle	True
solver	adam
tol	0.0001
validation_fraction	0.1
verbose	False
warm_start	False

Table A4. Random Forest Model.

Parameter	Value
bootstrap	True
<i>ccp_alpha</i>	0.0
criterion	squared_error
max_depth	None
max_features	log2
max_leaf_nodes	None
max_samples	None
min_impurity_decrease	0.0
min_samples_leaf	2
min_samples_split	2
min_weight_fraction_leaf	0.0
monotonic_cst	None
n_estimators	100
n_jobs	None
oob_score	False
random_state	42
verbose	0
warm_start	False

Table A5. XGBoost Model.

Parameter	Value
objective	reg & squarederror
base_score	None
booster	None
callbacks	None
colsample_bylevel	None
colsample_bynode	None
colsample_bytree	0.6
device	None
early_stopping_rounds	None
enable_categorical	False
eval_metric	None
feature_types	None
gamma	0.1
grow_policy	None
importance_type	None
interaction_constraints	None
learning_rate	0.2
max_bin	None
max_cat_threshold	None
max_cat_to_onehot	None
max_delta_step	None
max_depth	20
max_leaves	None
min_child_weight	None
missing	nan
monotone_constraints	None
multi_strategy	None
n_estimators	1000
n_jobs	None
num_parallel_tree	None
random_state	42
reg_alpha	0
reg_lambda	1.5

Table A5. *Cont.*

Parameter	Value
sampling_method	None
scale_pos_weight	None
subsample	0.8
tree_method	None
validate_parameters	None
verbosity	0

Table A6. Decision Tree Model.

Parameter	Value
ccp_alpha	0.0
criterion	squared_error
max_depth	20
max_features	sqrt
max_leaf_nodes	None
min_impurity_decrease	0.0
min_samples_leaf	10
min_samples_split	5
min_weight_fraction_leaf	0.0
monotonic_cst	None
random_state	42
splitter	best

References

1. Alioanei, C.; Popescu, N. AI-Based Solutions for Security and Resource Optimization in IoT Environments: A Systematic Review. *Information* **2025**, *16*, 841. [\[CrossRef\]](#)
2. Bensaoud, A.; Kalita, J.; Bensaoud, M. A Survey of Malware Detection Using Deep Learning. *Mach. Learn. Appl.* **2024**, *16*, 100546. [\[CrossRef\]](#)
3. Tayan, O.; Hassan, A.; Khankan, K.; Askool, S. Considerations for Adapting Higher Education Technology Courses for AI Large Language Models: A Critical Review of the Impact of ChatGPT. *Mach. Learn. Appl.* **2024**, *15*, 100513. [\[CrossRef\]](#)
4. Najjar, A.A.; Ashqar, H.I.; Darwish, O.; Hammad, E. Leveraging Explainable AI for LLM Text Attribution: Differentiating Human-Written and Multiple LLM-Generated Text. *Information* **2025**, *16*, 767. [\[CrossRef\]](#)
5. Kokkotis, C.; Moustakidis, S.; Swift, S.J.; Kontopidou, F.; Kavouras, I.; Doulamis, A.; Giannoukos, S. Artificial Intelligence and Machine Learning in the Diagnosis and Prognosis of Diseases Through Breath Analysis: A Scoping Review. *Information* **2025**, *16*, 968. [\[CrossRef\]](#)
6. Wardley, L.J.; Rajabi, E.; Amin, S.H.; Ramesh, M. A Machine Learning Approach Feature to Forecast the Future Performance of the Universities in Canada. *Mach. Learn. Appl.* **2024**, *16*, 100548. [\[CrossRef\]](#)
7. Albuquerque, P.C.C.; Cajueiro, D.O. Forecasting Political Voting: A High Dimensional Machine Learning Approach. *Mach. Learn. Appl.* **2025**, *22*, 100739. [\[CrossRef\]](#)
8. Kundu, T.; Pinsky, E. Predicting Daily Stock Price Directions with Deep Learning Models. *Mach. Learn. Appl.* **2025**, *22*, 100744. [\[CrossRef\]](#)
9. Afreen, N. Explainable and Faithful Educational Recommendations through Causal Language Modelling via Knowledge Graphs. In Proceedings of the RecSys '24: 18th ACM Conference on Recommender Systems, Association for Computing Machinery, Bari, Italy, 14–18 October 2024; pp. 1358–1360.
10. Chun-Hsiung Tseng, Hao-Chiang Koong Lin, A.C.W.H.; Lin, J.R. Personalized Programming Education: Using Machine Learning to Boost Learning Performance based on Students' Personality Traits. *Cogent Educ.* **2023**, *10*, 2245637. [\[CrossRef\]](#)
11. Dhananjaya, G.M.; Goudar, R.; Kulkarni, A.A.; Rathod, V.N.; Hukkeri, G.S. A Digital Recommendation System for Personalized Learning to Enhance Online Education: A Review. *IEEE Access* **2024**, *12*, 34019–34041. [\[CrossRef\]](#)
12. Sajja, R.; Sermet, Y.; Demir, I. End-to-End Deployment of the Educational AI Hub for Personalized Learning and Engagement: A Case Study on Environmental Science Education. *IEEE Access* **2025**, *13*, 55169–55186. [\[CrossRef\]](#)
13. Silva, L.; Mendes, A.; Gomes, A.; Fortes, G. What Learning Strategies are Used by Programming Students? A Qualitative Study Grounded on the Self-regulation of Learning Theory. *ACM Trans. Comput. Educ.* **2024**, *24*, 9. [\[CrossRef\]](#)

14. Mosquera, J.M.L.; Iturbide, J.Á.V.; Velasco, M.P.; Guerrero, V.A.B. Assessment of a Predictive Model for Academic Performance in a Small-Sized Programming Course. In Proceedings of the 2024 IEEE International Symposium on Computers in Education (SIIE), A Coruña, Spain, 19–21 June 2024; pp. 1–8.
15. Bucchiarone, A.; Martorella, T.; Frageri, D.; Colombo, D. PolyGloT: A Personalized and Gamified eTutoring System for Learning Modelling and Programming Skills. *Sci. Comput. Program.* **2024**, *231*, 103003. [[CrossRef](#)]
16. Al-Malki, L.; Meccawy, M. Investigating Students' Performance and Motivation in Computer Programming through a Gamified Recommender System. *Comput. Sch.* **2022**, *39*, 137–162. [[CrossRef](#)]
17. Zola, F.; Xabier. KINAITICS: Enhancing Cybersecurity Education Using AI-Based Tools and Gamification. In Proceedings of the 16th International Conference on Education Technology and Computers (ICETC), Porto, Portugal, 18–21 September 2024.
18. John, A.A.; Viendra. An Analysis of Machine Learning Algorithms for Predicting Student Performance. *Int. J. Adv. Res. Sci. Commun. Technol.* **2024**, *4*, 657–660. [[CrossRef](#)]
19. Qureshi, R.; Lokhande, P.S. A Comprehensive Review of Machine Learning techniques used for Designing An Academic Result Predictor and Identifying The Multi-Dimensional Factors Affecting Student's Academic Results. In Proceedings of the 2024 2nd DMIHER International Conference on Artificial Intelligence in Healthcare, Education and Industry (IDICAIEI), Wardha, India, 29–30 November 2024; pp. 1–6.
20. Pires, J.P.J.; Correia, F.B.; Gomes, A.; Borges, A.R.; Bernardino, J. Predicting Student Performance in Introductory Programming Courses. *Computers* **2024**, *13*, 219–219. [[CrossRef](#)]
21. Nabil, A.; Seyam, M.; Abou-Elfetouh, A. Prediction of Students' Academic Performance Based on Courses' Grades Using Deep Neural Networks. *IEEE Access* **2021**, *9*, 140731–140746. [[CrossRef](#)]
22. Alnoman, A. Will the Student Get an A Grade? Machine Learning-based Student Performance Prediction in Smart Campus. In Proceedings of the 2023 Advances in Science and Engineering Technology International Conferences (ASET), Dubai, United Arab Emirates, 20–23 February 2023; pp. 1–6.
23. Alhazmi, E.; Sheneamer, A. Early Predicting of Students Performance in Higher Education. *IEEE Access* **2023**, *11*, 27579–27589 [[CrossRef](#)]
24. Kinjal.; Parida, S.M.; Suthar, J.; Pande, S.D. Predicting Academic Success: A Comparative Study of Machine Learning and Clustering-Based Subject Recommendation Models. *EAI Endorsed Trans. Internet Things* **2024**, *10*, 1. [[CrossRef](#)]
25. Dixon, L.; Tang, T.; Darmasti, S. Early Birds Across Campuses: Predicting Student Performance Using Cross-Institutional Keystroke and Event Log Data. *Educ. Technol.* **2024**, *1*, 1–10.
26. Hoq, M.; Brusilovsky, P.; Akram, B. Explaining Explainability: Early Performance Prediction with Student Programming Pattern Profiling. *J. Educ. Data Min.* **2024**, *16*, 115–148.
27. Omri, A.; Bouallegue, S.; Feki, A.; Ford, R.; Ciftler, B.S.; Shikfa, A.; Khan, S.; Al-Naemi, S. Early Student Performance Prediction in Personalized Programming Education. In Proceedings of the 17th International Conference on Education Technology and Computers (ICETC 2025), Barcelona, Spain, 18–21 September 2025.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.